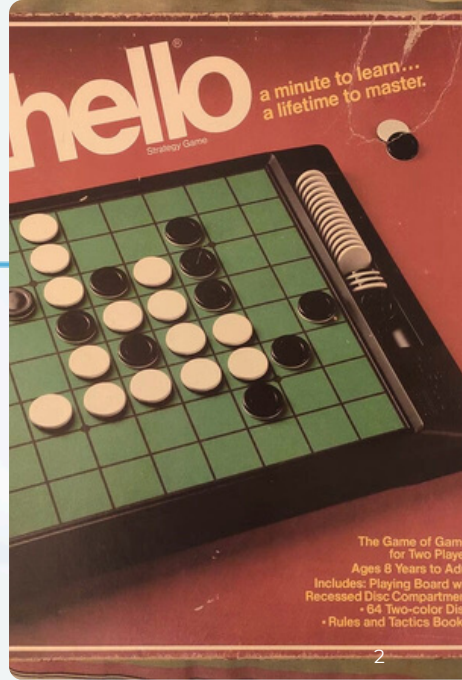


Intelligence Artificielle et Jeu Othello

Introdutcion



Plan

- Présentation du jeu
- Explorer les concepts de base de l'IA appliqués à Othello.
- Décrire les algorithmes et les heuristiques utilisés.



Le Jeu Othello

1

Invention par Lewis Waterman

Reversi a été créé par Lewis Waterman en 1883.

2

Popularisation dans les années 1970

Othello a été commercialisé au Japon et est devenu extrêmement populaire.

Origine japonaise

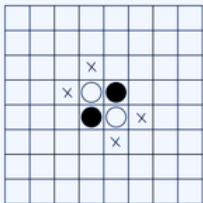
Othello est une variante du jeu Reversi inventé en Angleterre à la fin du XIXe siècle.

Jeu de réflexion

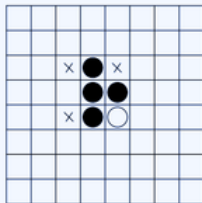
Il se joue sur un plateau de 8x8 cases avec des pions bicolores.

Règles et objectif du jeu

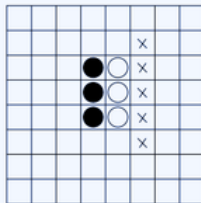
Plateau	8x8 cases
Pions	Deux couleurs : noir et blanc
Objectif	Avoir le plus de pions de sa couleur à la fin du jeu



début



noir joue



blanc joue

Pourquoi l'IA est intéressante pour jouer Othello :

1 Règles simples, stratégie profonde :

Chaque mouvement change significativement l'état du jeu, ce qui rend les prédictions complexes.

2 Complexité computationnelle :

La nécessité d'anticiper les coups de l'adversaire ajoute une couche de complexité.

3 Défi d'adaptabilité :

Variété des tactiques : démontre capacité d'IA à s'adapter dynamiquement.

Fondements Théoriques de l'IA :

Définition de l'IA

La capacité des machines à imiter des fonctions cognitives humaines telles que l'apprentissage et la résolution de problèmes.

Types d'IA

L'IA faible est spécialisée dans des tâches spécifiques (comme jouer à Othello), tandis que l'IA forte (ou générale) peut effectuer toute tâche cognitive humaine.

IA FAIBLE 	 IA FORTE
• Google Traduction :	• Jarvis (Iron Man) :
• Autopilot de Tesla :	• Ava (Ex Machina) :
• ChatGPT :	• Skynet (Terminator)

Sous-disciplines de l'intelligence artificielle :

Algorithmique

- Pathfinding (Recherche de chemin) : A*
- Génération procédurale de contenu

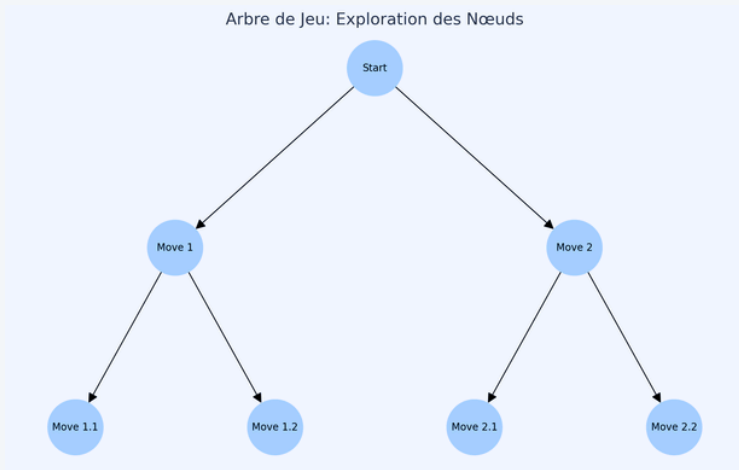
théorie des jeux.

- Équilibre des mécanismes de jeu
- Conception de mécaniques de jeu

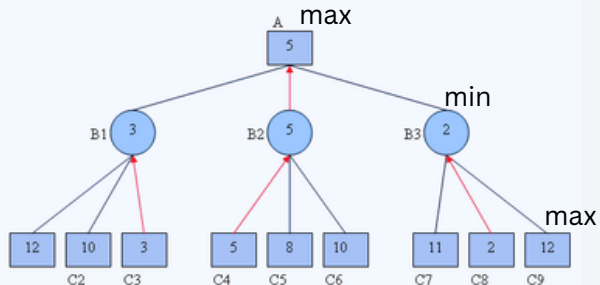
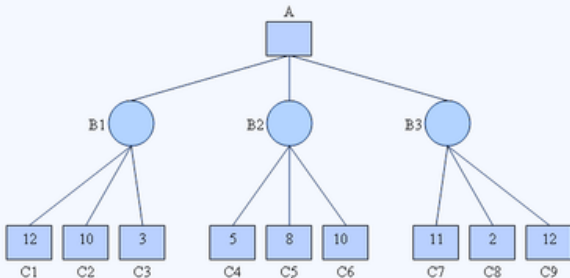
apprentissage automatique

- Réalisme et immersion
- Analyse de données de jeu

Algorithmes de Recherche Arborescente :



Algorithme Minimax:



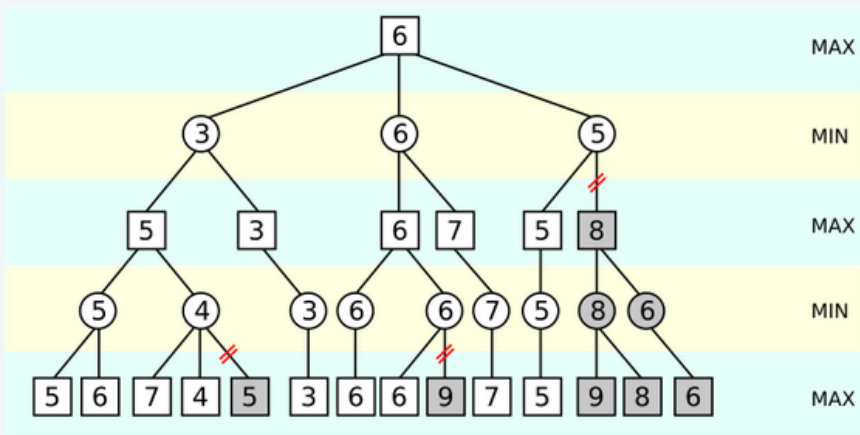
fonction minimax(noeud, profondeur, joueurMaximisant) est
si profondeur == 0 ou noeud est un nœud terminal alors
retourner la valeur heuristique de noeud

si joueurMaximisant alors
valeur := $-\infty$
pour chaque enfant de noeud faire
valeur := max(valeur, minimax(enfant, profondeur - 1, FALSE))

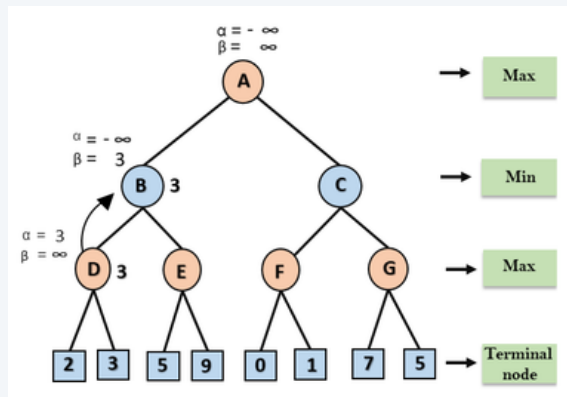
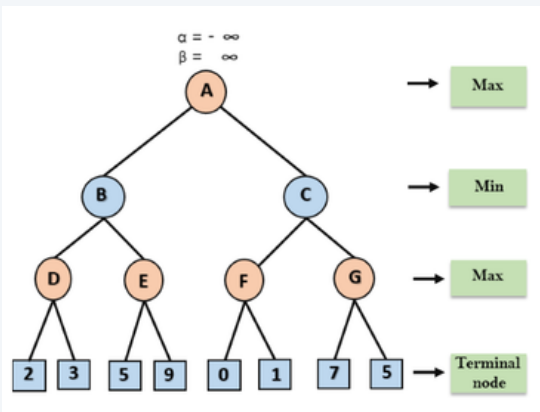
sinon (* joueur minimisant *)
valeur := $+\infty$
pour chaque enfant de noeud faire
valeur := min(valeur, minimax(enfant, profondeur - 1, TRUE))

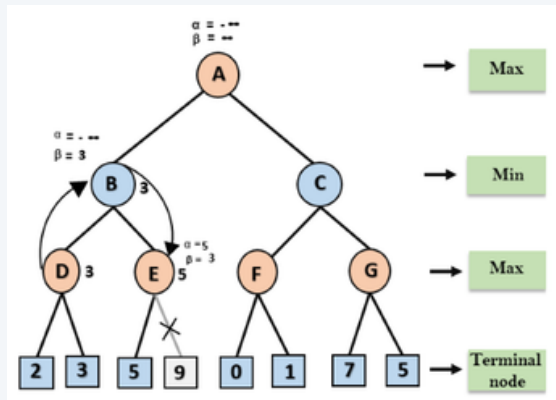
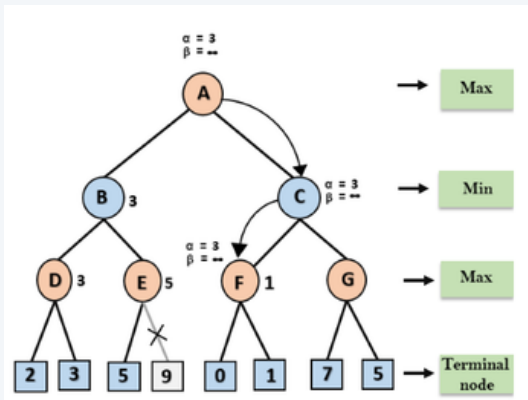
retourner valeur

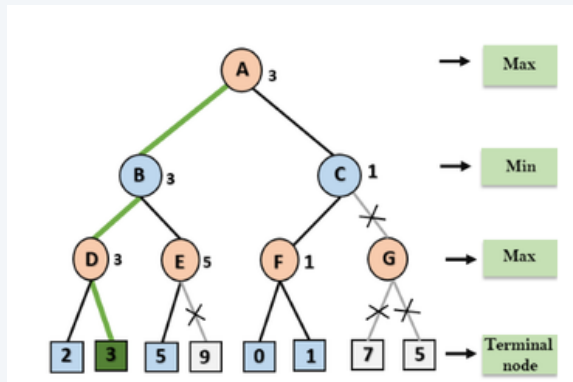
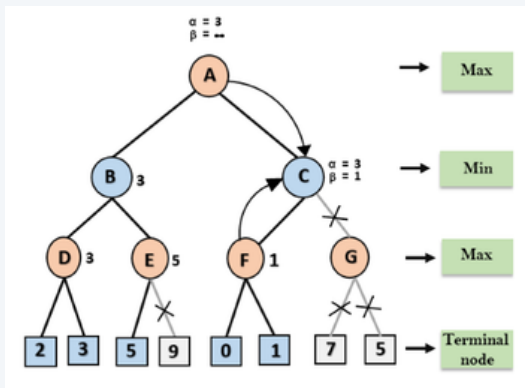
Algorithme Minimax:



Élagage AlphaBeta :







Heuristiques et Fonctions d'Évaluation :

Comment l'IA évalue-t-elle les positions sur le plateau ?

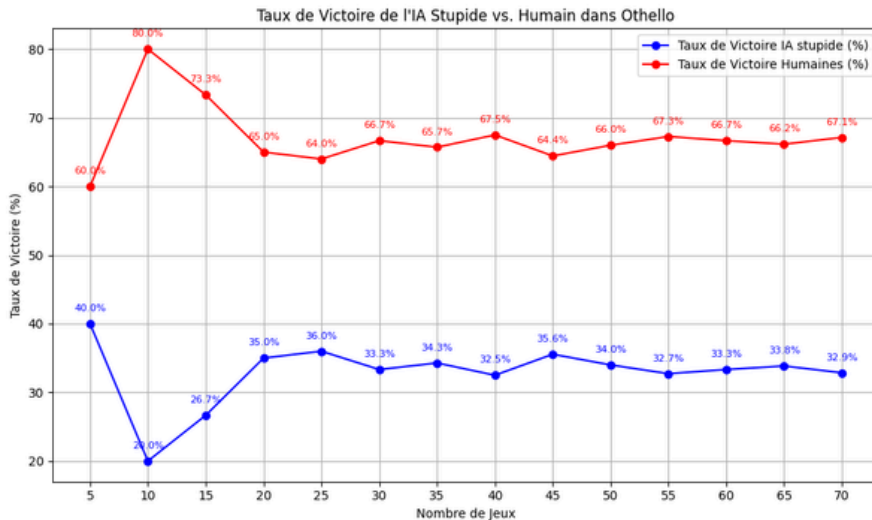
Importance des heuristiques

- Les heuristiques simplifient le processus de recherche en fournissant des approximations rapides des valeurs des positions.
- Elles sont cruciales pour évaluer les positions intermédiaires sans devoir explorer tous les coups possibles jusqu'à la fin du jeu.

Exemples de critères d'évaluation

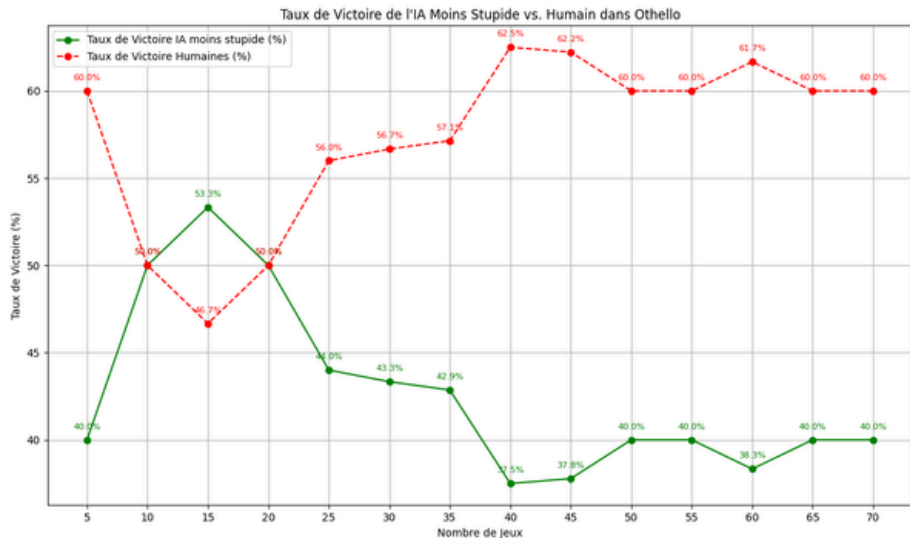
- Nombre de pions :
 - Compter le nombre de pions d'une couleur spécifique sur le plateau.
- Stabilité des pions :
 - Identifier les pions qui ne peuvent pas être retournés par l'adversaire.
- Mobilité des pions :
 - Compter le nombre de mouvements possibles pour un joueur.

IA stupide : Choisit un mouvement aléatoire

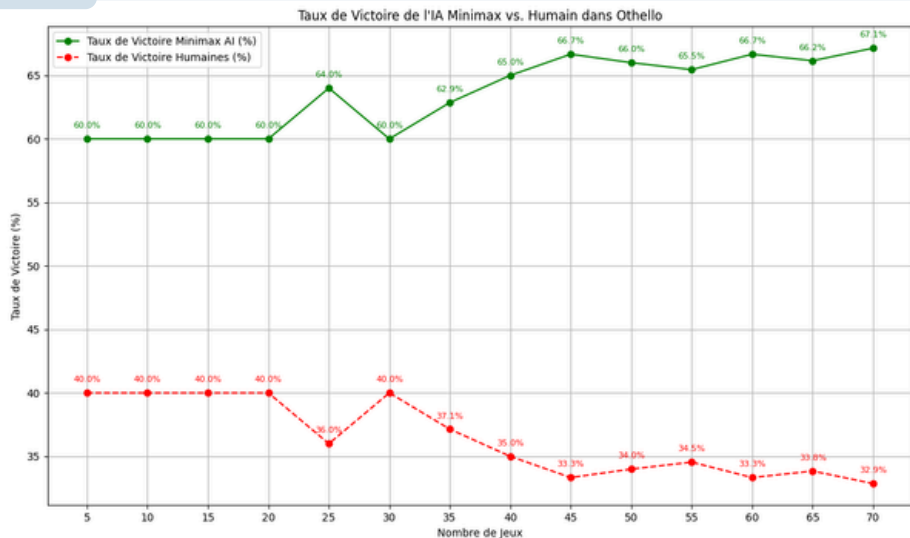




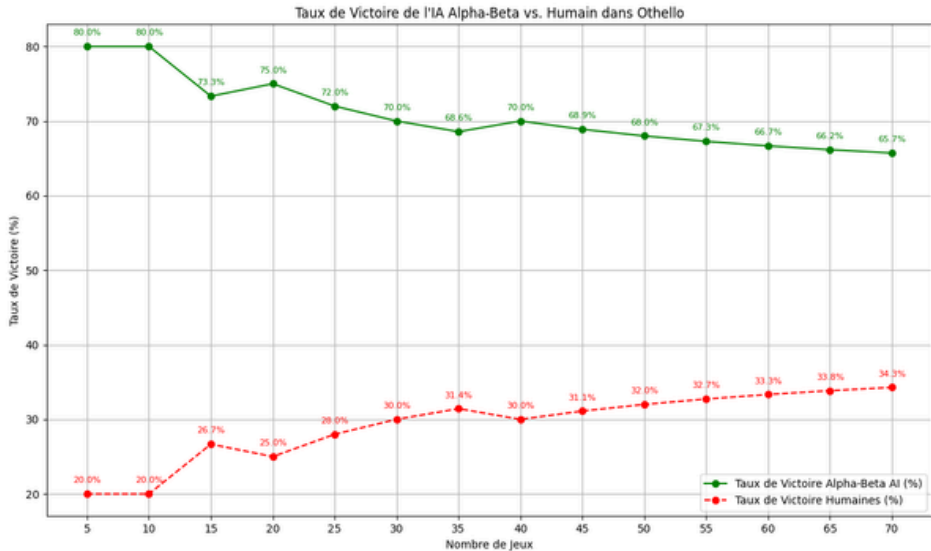
IA Moins Stupide : aide d'une heuristique "Dumb" : dumbScore()



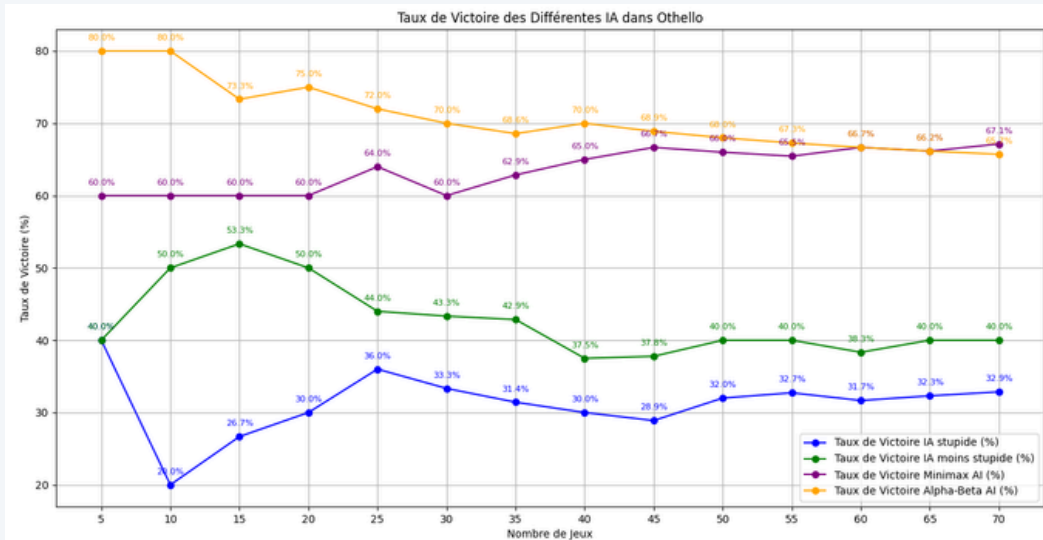
IA MINIMAX:

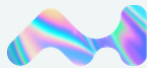
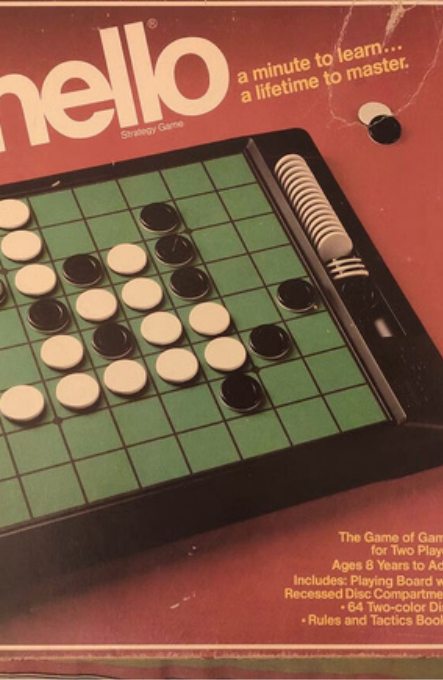


IA Alpha-Beta :



Comparaison entre les IA





Conclusion



*merci pour
votre
attention*



Annexes

Affichage :

```
import tkinter as tk
import numpy as np
import math
import random
import pandas as pd
import matplotlib.pyplot as plt

# Définir le jeu de données simulé pour les différentes IA avec un décalage de 5 jeux
data = {
    'Nombre de Jeux': [i for i in range(5, 75, 5)],
    'Victoires IA stupide': [2, 2, 4, 6, 9, 10, 11, 12, 13, 16, 18, 19, 21, 23],
    'Victoires IA moins stupide': [2, 5, 8, 10, 11, 13, 15, 15, 17, 20, 22, 23, 26, 28],
    'Victoires Minimax AI': [3, 6, 9, 12, 16, 18, 22, 26, 30, 33, 36, 40, 43, 47],
    'Victoires Alpha-Beta AI': [4, 8, 11, 15, 18, 21, 24, 28, 31, 34, 37, 40, 43, 46]
}

# Créer un DataFrame
df = pd.DataFrame(data)

# Calculer les taux de victoire pour chaque IA
df['Taux de Victoire IA stupide (%)'] = (df['Victoires IA stupide'] / df['Nombre de Jeux']) * 100
df['Taux de Victoire IA moins stupide (%)'] = (df['Victoires IA moins stupide'] / df['Nombre de Jeux']) * 100
df['Taux de Victoire Minimax AI (%)'] = (df['Victoires Minimax AI'] / df['Nombre de Jeux']) * 100
df['Taux de Victoire Alpha-Beta AI (%)'] = (df['Victoires Alpha-Beta AI'] / df['Nombre de Jeux']) * 100

# Tracer les données
plt.figure(figsize=(12, 8))

# IA stupide
plt.plot(df['Nombre de Jeux'], df['Taux de Victoire IA stupide (%)'], marker='o', linestyle='--', color='b', label='Taux de Victoire IA stupide')

# Ajouter les annotations pour IA stupide
for i in range(len(df)):
    plt.annotate(f"{df['Taux de Victoire IA stupide (%)'][i]:.1f}%",
                (df['Nombre de Jeux'][i], df['Taux de Victoire IA stupide (%)'][i]),
                textcoords="offset points", xytext=(0,10), ha='center', fontsize=8, color='blue')
```

```

# IA moins stupide
plt.plot(df['Nombre de Jeux'], df['Taux de Victoire IA moins stupide (%)'], marker='o', linestyle='--', color='g', label='Taux de Victoire IA

# Ajouter les annotations pour IA moins stupide
for i in range(len(df)):
    plt.annotate(f"{df['Taux de Victoire IA moins stupide (%)'][i]:.1f}%",
                (df['Nombre de Jeux'][i], df['Taux de Victoire IA moins stupide (%)'][i]),
                textcoords="offset points", xytext=(0,10), ha='center', fontsize=8, color='green')

# Minimax AI
plt.plot(df['Nombre de Jeux'], df['Taux de Victoire Minimax AI (%)'], marker='o', linestyle='--', color='purple', label='Taux de Victoire Mir

# Ajouter les annotations pour Minimax AI
for i in range(len(df)):
    plt.annotate(f"{df['Taux de Victoire Minimax AI (%)'][i]:.1f}%",
                (df['Nombre de Jeux'][i], df['Taux de Victoire Minimax AI (%)'][i]),
                textcoords="offset points", xytext=(0,10), ha='center', fontsize=8, color='purple')

# Alpha-Beta AI
plt.plot(df['Nombre de Jeux'], df['Taux de Victoire Alpha-Beta AI (%)'], marker='o', linestyle='--', color='orange', label='Taux de Victoire

# Ajouter les annotations pour Alpha-Beta AI
for i in range(len(df)):
    plt.annotate(f"{df['Taux de Victoire Alpha-Beta AI (%)'][i]:.1f}%",
                (df['Nombre de Jeux'][i], df['Taux de Victoire Alpha-Beta AI (%)'][i]),
                textcoords="offset points", xytext=(0,10), ha='center', fontsize=8, color='orange')

plt.title('Taux de Victoire des Différentes IA dans Othello')
plt.xlabel('Nombre de Jeux')
plt.ylabel('Taux de Victoire (%)')
plt.xticks(df['Nombre de Jeux'])
plt.grid(True)
plt.legend(fontsize="10", loc='lower right')

plt.tight_layout()
plt.show()

```

```

#IA stupide : Choisit un mouvement aléatoire
def dumbMove(self):
    #Générez tous les mouvements possibles
    choices = []
    for x in range(8):
        for y in range(8):
            if valid(self.array,self.player,x,y):
                choices.append([x,y])
    #Choisit un mouvement aléatoire
    dumbChoice = choice(choices)
    self.arrayMove(dumbChoice[0],dumbChoice[1])

# IA Moins stupide - Choisit un mouvement en fonction de ce qui lui rapportera le plus de pièces au prochain tour.
def slightlyLessDumbMove(self):
    boards = []
    choices = []
    for x in range(8):
        for y in range(8):
            if valid(self.array,self.player,x,y):
                test = move(self.array,x,y)
                boards.append(test)
                choices.append([x,y])

    #Détermine le meilleur score en fonction des tableaux générés précédemment et d'une heuristique "Dumb": dumbScore()
    bestScore = -float("inf")
    bestIndex = 0
    for i in range(len(boards)):
        score= dumbScore(boards[i],self.player)
        if score>bestScore:
            bestIndex=i
            bestScore = score
    self.arrayMove(choices[bestIndex][0],choices[bestIndex][1])

```

```

def minimax(self, node, depth, maximizing):
    global nodes
    nodes += 1
    boards = []
    choices = []

    for x in range(8):
        for y in range(8):
            if valid(self.array, self.player, x, y):
                test = move(node, x, y)
                boards.append(test)
                choices.append([x, y])

    if depth == 0 or len(choices) == 0:
        return ([decentHeuristic(node, 1 - maximizing), node])

    if maximizing:
        bestValue = -float("inf")
        bestBoard = []
        for board in boards:
            val = self.minimax(board, depth - 1, 0)[0]
            if val > bestValue:
                bestValue = val
                bestBoard = board
        return ([bestValue, bestBoard])

```

```

else:
    bestValue = float("inf")
    bestBoard = []
    for board in boards:
        val = self.minimax(board, depth - 1, 1)[0]
        if val < bestValue:
            bestValue = val
            bestBoard = board
    return ([bestValue, bestBoard])

```

```

def alphaBeta(self,node,depth,alpha,beta,maximizing):
    global nodes
    nodes += 1
    boards = []
    choices = []

    for x in range(8):
        for y in range(8):
            if valid(self.array,self.player,x,y):
                test = move(node,x,y)
                boards.append(test)
                choices.append([x,y])

    if depth==0 or len(choices)==0:
        return ([finalHeuristic(node,maximizing),node])

    if maximizing:
        v = -float("inf")
        bestBoard = []
        bestChoice = []
        for board in boards:
            boardValue = self.alphaBeta(board,depth-1,alpha,beta,0)[0]
            if boardValue>v:
                v = boardValue
                bestBoard = board
                bestChoice = choices[boards.index(board)]
            alpha = max(alpha,v)
            if beta <= alpha:
                break
        return([v,bestBoard,bestChoice])

```

```

else:
    v = float("inf")
    bestBoard = []
    bestChoice = []
    for board in boards:
        boardValue = self.alphaBeta(board,depth-1,alpha,beta,1)[0]
        if boardValue<v:
            v = boardValue
            bestBoard = board
            bestChoice = choices[boards.index(board)]
        beta = min(beta,v)
        if beta<=alpha:
            break
    return([v,bestBoard,bestChoice])

```

#heuristique simple. Compare le nombre de chaque tuile.

```
def dumbScore(array,player):  
    score = 0  
    #Set player and opponent colours  
    if player==1:  
        colour="b"  
        opponent="w"  
    else:  
        colour = "w"  
        opponent = "b"  
  
    for x in range(8):  
        for y in range(8):  
            if array[x][y]==colour:  
                score+=1  
            elif array[x][y]==opponent:  
                score-=1  
    return score
```

```

def slightlyLessDumbScore(array,player):
    score = 0
    #Définir les couleurs du joueur et de l'adversaire
    if player==1:
        colour="b"
        opponent="w"
    else:
        colour = "w"
        opponent = "b"
    for x in range(8):
        for y in range(8):
            #Tuiles normales valant 1
            add = 1
            #Tuiles de bord d'une valeur de 3
            if (x==0 and 1<y<6) or (x==7 and 1<y<6) or (y==0 and 1<x<6) or (y==7 and 1<x<6):
                add=3
            #Tuiles d'angle d'une valeur de 5
            elif (x==0 and y==0) or (x==0 and y==7) or (x==7 and y==0) or (x==7 and y==7):
                add = 5
            #Ajouter ou soustraire la valeur de la tuile correspondant à la couleur
            if array[x][y]==colour:
                score+=add
            elif array[x][y]==opponent:
                score-=add
    return score

```

```
def decentHeuristic(array,player):
    score = 0
    cornerVal = 25
    adjacentVal = 5
    sideVal = 5
    #Définir les couleurs du joueur et de l'adversaire
    if player==1:
        colour="b"
        opponent="w"
    else:
        colour = "w"
        opponent = "b"
    #Parcourez toutes les tuiles
    for x in range(8):
        for y in range(8):
            add = 1
```

```
#Tuiles de bord d'une valeur de -3
if (x==0 and y==1) or (x==1 and 0<=y<=1):
    if array[0][0]==colour:
        add = sideVal
    else:
        add = -adjacentVal

elif (x==0 and y==6) or (x==1 and 6<=y<=7):
    if array[7][0]==colour:
        add = sideVal
    else:
        add = -adjacentVal
```



```

elif (x==7 and y==1) or (x==6 and 0<=y<=1):
    if array[0][7]==colour:
        add = sideVal
    else:
        add = -adjacentVal

elif (x==7 and y==6) or (x==6 and 6<=y<=7):
    if array[7][7]==colour:
        add = sideVal
    else:
        add = -adjacentVal

#Tuiles de bord d'une valeur de 3
elif (x==0 and 1<y<6) or (x==7 and 1<y<6) or (y==0 and 1<x<6) or (y==7 and 1<x<6):
    add=sideVal
#Tuiles de coin d'une valeur de 15
elif (x==0 and y==0) or (x==0 and y==7) or (x==7 and y==0) or (x==7 and y==7):
    add = cornerVal
#Ajouter ou soustraire la valeur de la tuile correspondant à la couleur
if array[x][y]==colour:
    score+=add
elif array[x][y]==opponent:
    score-=add

return score

```

#Séparer l'utilisation des heuristiques pour le début, le milieu et la fin du jeu.

```
def finalHeuristic(array,player):  
    if moves<=8:  
        numMoves = 0  
        for x in range(8):  
            for y in range(8):  
                if valid(array,player,x,y):  
                    numMoves += 1  
        return numMoves+decentHeuristic(array,player)  
    elif moves<=52:  
        return decentHeuristic(array,player)  
    elif moves<=58:  
        return slightlyLessDumbScore(array,player)  
    else:  
        return dumbScore(array,player)
```